

Synthetic data

Manufacturing training data on purpose: generation, filtering, and the collapse you keep hearing about

Your SFT plan (Chapter 23) calls for 200,000 high-quality instruction–response pairs in Spanish and Portuguese. The open datasets are English translated badly, the vendors quote dollars per pair with six-week lead times, and your annotation budget covers a twentieth of the need. Meanwhile the strongest model in your cluster can draft a competent instruction pair in two seconds for a hundredth of a cent. The obvious move has a name that used to be an insult — *training on model outputs* — and it now underwrites the frontier: Phi-4 pre-trained on ~400B tokens of curated synthetic text and mostly synthetic post-training data¹; Nemotron-4's alignment pipeline was **98% synthetic**²; Llama 3's post-training leaned on model-generated, rejection-sampled data at millions-of-examples scale³; and Chapter 26's reasoning distillates were built from 800K sampled traces. The question stopped being *whether* to manufacture data years ago. The question is the one this chapter answers: how to manufacture it so that quality goes *up* — and how to avoid the well-documented ways it goes down.

One framing organizes everything. Synthetic data is a **factory**, and like any factory it has three stations: *generation* (what raw material, shaped how), *quality control* (filtering, verification, rejection — where most of the value is actually created), and *compliance* (licenses, provenance, contamination). Teams fail at station two and three far more often than at station one, because generation is the fun part and the other two are discipline. The chapter walks the stations in order, then confronts the collapse question directly, because you will be asked about it in every review.

27.1 Generation: the design space is prompts × seeds × verifiers

Every generation recipe answers three questions: where does *variety* come from (the seed), what does the *generator* do with it, and can the output be *checked* by something other than taste. The named methods you will meet are points in that space, and knowing the space matters more than knowing the names:

THE DESIGN SPACE — every named method is a choice in each column

1 · seed (variety source)

- a few hand-written examples
- real corpus documents
- personas (1B synthetic identities)
- taxonomies of topics × skills
- existing weak data, to be evolved
- nothing: the aligned model's own chat template (Magpie)

seed diversity ⇒ output diversity

2 · generator pattern

- imitate: more like these seeds
- evolve: rewrite harder / deeper / broader (Evol-Instruct)
- transform: rephrase real text (WRAP), translate + localize
- teach: textbook-style exposition of a topic list (Phi, Cosmopedia)
- solve: problems → traces (Ch. 26)

the prompt IS the product spec

3 · verifier (can it be checked?)

- executable: run the code, run the tests — binary truth
- checkable: math answer, schema, exact-match QA
- judgeable: LLM judge + rubric (biases apply — §27.2)
- unverifiable: style, chat, opinion — filter hard, trust less

volume goes where you can verify

Named methods, placed: Self-Instruct = examples × imitate × judgeable · Evol-Instruct = weak data × evolve × judgeable · Persona Hub = personas × imitate/solve × mixed · Phi/Cosmopedia = taxonomy × teach × judgeable · R1 traces = problems × solve

Figure 27.1 — Three choices, not twenty methods. Seeds buy diversity, the generator pattern is written as a prompt, and the verifier column decides how much you can trust the result at scale. The single most reliable rule in this chapter lives in the third column: put your volume where verification is mechanical.

A tour of the space's landmarks, briefly, since each is one prompt-shape. **Self-Instruct** started it: 175 hand-written seed tasks, a loop that asks the model for new tasks in the same format, and filters — 52K instructions from almost nothing, and the template for everything since.⁴ **Evol-Instruct** replaces "more like this" with rewriting pressure — *make this instruction require deeper reasoning; add a rare constraint; widen the topic* — evolving easy data into hard data, which matters because difficulty is exactly what naive generation lacks.⁵ **Persona-driven** generation attacks the diversity ceiling from the seed side: condition every sample on a different synthetic person ("a nurse in Arequipa who tracks medication schedules"), and a billion personas manufacture a billion contexts — the Persona Hub result showed this lifts measured diversity dramatically at constant prompt machinery.⁶ **Magpie** is the elegant trick at the "no seed at all" corner: hand an aligned chat model only its own template prefix and let it complete *both* the user turn and the response — self-extraction of the instruction distribution the model was aligned on.⁷ For *pre-training-scale* synthesis the workhorses are transform and teach: **WRAP-style rephrasing** rewrites crawl text into cleaner registers (turning one real corpus into several stylistic views of itself)⁸, and **textbook-style** generation walks a topic taxonomy and writes expositions — Cosmopedia's open reproduction of the Phi recipe shipped 25B tokens this way.⁹

Because "the prompt is the product spec," it deserves the same care as a config file. A production template pins the three columns explicitly — seed slot, pattern instructions, and the output structure the verifier will parse:

```
# persona-seeded, natively-Spanish instruction generation – one tracked template
TEMPLATE_ID = "es-persona-instr-v3"          # provenance: every sample carries this
PROMPT = ""Eres {persona}.                  # seed slot: 1 of 50K personas, rotated
Escribe UNA tarea que tú, con tu trabajo y contexto, le pedirías a un
asistente de IA – específica, realista, con los detalles que tú conocerías.
Luego escribe la respuesta ideal: correcta, completa, y en el registro
que tú esperarías.
Formato estricto:                            # structure: what the parser verifies
### TAREA\n...\n### RESPUESTA\n...\n""

for persona in sample(personas_es, k=BATCH):    # seed rotation = diversity
```

```

out = generator.chat(PROMPT.format(persona=persona), temperature=1.0)
task, resp = parse_sections(out) # malformed -> funnel stage
3
emit({"template": TEMPLATE_ID, "persona_id": persona.id,
     "task": task, "response": resp}) # provenance travels with
data
# the same template family exists for pt-BR – generated natively, never translated

```

And one counterweight before you industrialize: **volume is not the goal — coverage is**. The LIMA result made the point sharply: 1,000 meticulously curated examples aligned a 65B base model to a quality that millions of mediocre pairs did not reach, because SFT mostly teaches *format and register*, which saturate fast, while knowledge comes from pre-training.¹⁶ The synthesis factory's advantage over hand-curation is not that it makes more — it is that it makes *coverage* cheap: a thousand personas × a hundred task types × two languages is a curriculum no annotation team can afford, at volumes small enough that your judges and auditors can still keep up. Generate for the grid, not for the count.

The pattern to internalize from the tour: **variety is engineered at the seed, difficulty at the generator, trust at the verifier**. When a synthetic set disappoints, diagnose by column — repetitive data is a seed problem, easy data is a generator problem, wrong data is a verifier problem — and fix that column, not "the prompt" in general.

27.2 Filtering and rejection sampling: where the quality is made

Generation produces candidates; the factory's value is created by what you throw away. The teams that ship good synthetic data run a **funnel**, and the numbers are humbling: keeping a third of what you generate is normal, and keeping much more than that usually means your filters are asleep, not that your generator is brilliant.

THE FUNNEL — illustrative rates for an instruction-data run (yours will differ; measure them)

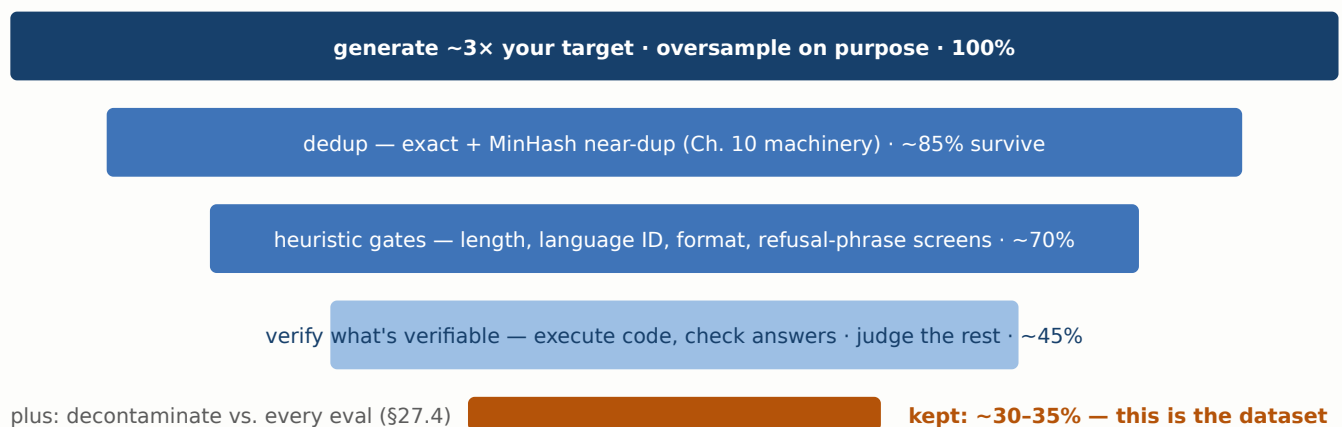


Figure 27.2 — Plan the funnel before the generation run. Oversampling $\sim 3\times$ and keeping a third beats generating exactly what you need and keeping it all — the same tokens of teacher compute, radically different dataset. Instrument every stage's survival rate; a stage that suddenly keeps 95% has usually broken, not improved.

Two of the funnel's stages deserve their mechanics spelled out. **Verification** is the third column of Figure 27.1 cashing out: for code, run it — unit tests convert generation into search, and Llama 3's team describes exactly this loop

(generate, execute, keep what passes, iterate on what fails)³; for math, check the final answer against ground truth or a solver; for structured outputs, validate the schema. Every sample that survives a mechanical check is worth several that survived a vibe check. **Rejection sampling** generalizes this when the check is a score rather than a binary: draw n candidates per prompt (8–16 is common), score them with a reward model or judge, keep the best (or all above a bar) — best-of- n selection concentrates quality for one extra factor of teacher compute, and it was a workhorse of Llama 3's post-training data at scale.³

Where no mechanical check exists, the scorer is an **LLM judge**, and you must treat the judge as an instrument with known systematic errors, because it has them: *position bias* (prefers the first answer shown — always score both orders), *length bias* (mistakes longer for better — cap or normalize), and *self-preference* (rates its own family's style higher — use a judge from a different family than the generator when you can).¹⁰ The working harness is short:

```
# rejection sampling with a debiased judge – the funnel's last two stages
JUDGE_RUBRIC = """Score the RESPONSE to the INSTRUCTION from 1-5:
correctness and completeness (3), clarity (1), instruction-following (1).
A fluent response that dodges the instruction scores at most 2.
Output only the integer."""

def keep_best(prompt, generator, judge, n=8, bar=4):
    cand = [generator.chat(prompt, temperature=1.0) for _ in range(n)]
    scored = []
    for c in cand:
        s1 = int(judge.chat(f"{JUDGE_RUBRIC}\nINSTRUCTION: {prompt}\nRESPONSE: {c}"))
        s2 = int(judge.chat(f"{JUDGE_RUBRIC}\nRESPONSE: {c}\nINSTRUCTION: {prompt}"))
        scored.append((min(s1, s2), c))          # order-swap: keep the harsher
    read
    best = max(scored)
    return best[1] if best[0] >= bar else None    # below bar: the prompt goes
back                                           # to the generator, not the
dataset
# judge from a DIFFERENT model family than the generator; audit 100 samples by
hand
# per run – the judge drifts when the generator's style does.
```

27.3 Model collapse: the real risk, and the version that isn't

You will be asked about the Nature paper: models trained on the outputs of models degrade, generation over generation, forgetting the tails of the distribution first — *model collapse*.¹¹ The result is real, and it is also routinely misread as "synthetic data poisons models," which the frontier's own practice plainly contradicts. The resolution is in the experimental setup: collapse experiments **replace** the data — each generation trains *only* on the previous model's output, a photocopy of a photocopy, and the tails (rare facts, rare styles, minority languages) blur first because sampling

under-represents them and each iteration compounds the loss. Follow-up work showed the knife-edge cleanly: when synthetic data **accumulates alongside** real data instead of replacing it, the degradation largely disappears.¹²

The tail-forgetting mechanism is worth one line of arithmetic, because it explains *why* replacement collapses and accumulation doesn't. A token or fact with probability $p = 0.01$ fails to appear in $n = 100$ samples with probability $(1 - p)^{100} \approx 0.366$ — so in a replace-style pipeline, each generation has a one-in-three chance of never seeing that fact *at all*, after which the next model's p is zero and the loss is permanent. Rare facts, rare registers, and minority languages sit exactly in that p -range, which is why they blur first — and why the fix is structural rather than clever: keep the real corpus (where p never degrades) in every mix, and over-seed the tails you care about instead of hoping sampling finds them.

COLLAPSE IS A PIPELINE SHAPE, NOT A MATERIAL PROPERTY (schematic)

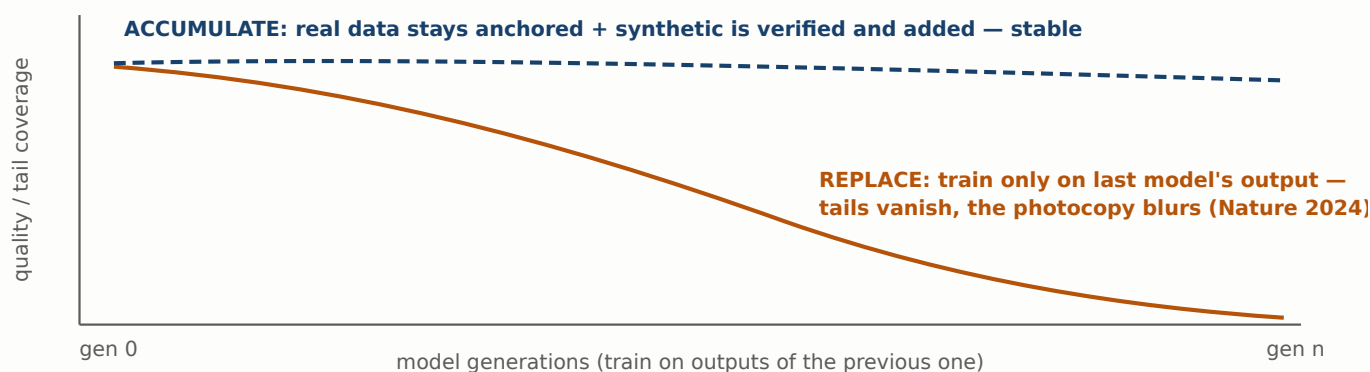


Figure 27.3 — Replace vs. accumulate. The published collapse results train each generation *only* on the previous model's outputs; keeping the real corpus in the mix and verifying what synthetic joins it removes most of the effect.¹¹¹² Schematic curves — the operational rules below are what you actually enforce.

So the practice distills to four operational rules, and they double as your answer in the review meeting. **Anchor:** real data never leaves the mix; synthetic augments, it does not replace. **Verify:** prefer generation where Figure 27.1's third column is mechanical, so errors can't compound invisibly. **Diversify at the seed:** collapse is tail-forgetting, and persona/taxonomy seeding is tail life-support — for a bilingual model this includes seeding in *both* languages natively rather than generating English and translating, because translationese is itself a collapsed register. **Monitor the tails:** track type–token ratios, topic entropy, and rare-language shares of the mix per batch of synthetic data, so drift is a dashboard line, not an autopsy finding.

27.4 Licensing, provenance, and contamination

The compliance station is short but it is a gate, not a suggestion, and Chapter 26 already opened its two hardest doors.

Licenses: if the generator is a frontier API, its terms likely restrict training competing models on outputs; if it is open-weight, licenses differ meaningfully (some bless synthetic data generation explicitly — Nemotron's permissive release was pitched precisely as "a generator you're allowed to use"²). Decide the teacher *with* whoever owns legal risk, before the run. **Provenance:** every synthetic sample carries metadata — generator model and version, prompt template id, seed, judge scores, verification result — stored with the data (Chapter 10's versioning discipline). This is what lets you later remove one bad template's output surgically instead of burning the dataset, and it is what auditors and license

reviews will ask for. **Contamination:** a teacher that has memorized public benchmarks will emit them, verbatim or paraphrased, into your training set — inflating your evals from inside. Decontaminate synthetic data against every suite you report (n-gram and embedding overlap, Chapter 9), and treat a sudden benchmark jump after a synthetic batch as a contamination alarm first and a triumph second.

One more product the factory ships, pointing forward: **synthetic preferences**. Rank two model responses with a judge (or better, with a verifier) and you have a DPO pair (Chapter 24) without a human in the loop — the Zephyr recipe's dDPO, and the bulk of Nemotron's 98%.² Every judge caveat from §27.2 applies squared, because preference data *defines* the reward landscape: a length-biased judge here doesn't just admit verbose samples, it teaches the model that verbosity is what humans want. Verify where possible, audit relentlessly where not, and re-read §28.3's warning about preference signals that never punish refusals — synthetic preference pipelines are exactly where that failure gets manufactured at scale.

27.5 What the big labs do

Table 27.1 — Synthetic data in shipped pipelines.

SYSTEM	WHERE SYNTHETIC SITS	RECIPE	SCALE / RESULT
Phi-4 ¹	pre-training core	taxonomy-seeded "textbook" exposition + exercises, multi-stage curation, self-revision	~400B synthetic tokens; 14B model competing far above its size on reasoning
Cosmopedia (open) ⁹	pre-training	open reproduction of the Phi recipe: web-anchored topics × audiences × styles	25B tokens, 30M documents — the public reference implementation
Nemotron-4 340B ²	post-training	generator + judge + preference pipeline, released as a permissively-licensed kit	98% of alignment data synthetic
Llama 3 ³	post-training	rejection sampling at scale; execution-verified code loops; model-judged quality gates	millions of SFT examples, majority model-generated
Qwen2.5-1M ¹³	mid-training	synthetic long-range dependencies — document-scale FIM, cross-window retrieval (Ch. 21)	the data half of the 1M context recipe
DeepSeek R1 distillates ¹⁴	post-training	reasoning traces, answer-verified (= Ch. 26's black-box distillation)	800K samples; the 2025 wave of small reasoners

SYSTEM	WHERE SYNTHETIC SITS	RECIPE	SCALE / RESULT
UltraChat / Zephyr-era ¹⁵	post-training	early large-scale multi-turn synthesis (1.5M dialogues), heavy post-filtering	proved open models could be aligned on synthetic chat alone

Note what the table quietly says about *where* synthetic data pays. At pre-training scale it works when anchored to structure (taxonomies, real web text to rephrase) and used *alongside* the crawl, not instead of it. In mid-training it manufactures properties the natural corpus lacks (long-range dependency). In post-training it is simply how the work gets done now — the debate ended when the pipelines went 90%+ synthetic and the models kept getting better. Every row also passed through a funnel like Figure 27.2's; none of these teams ships raw generator output.

27.6 Practical considerations

Cost the factory like a serving problem. Generation is teacher inference, so Chapter 13's economics apply: batch it, cache shared prompt prefixes (taxonomy stems and few-shot headers repeat millions of times — prefix caching pays for itself immediately), quantize the generator, and prefer your own hardware for volume (the arithmetic from Ch. 26's trace-budget section: billions of tokens through an API is a purchase order; through an idle training cluster it is a weekend). Judges add a second inference bill that people forget to budget — order-swapped scoring doubles it again.

Run the numbers for a real order. Target: 200K kept Spanish/Portuguese instruction pairs. The funnel says generate ~600K candidates; at ~700 tokens per candidate (prompt + task + response) that is ~0.4B generated tokens. Best-of-8 with order-swapped judging adds ~16 judge calls per *kept* sample at ~1K tokens each — ~3.2B judge tokens, the quietly dominant line item. Total $\approx$ 3.6B tokens of inference: on a self-hosted 70B-class generator sustaining $\sim 10^4$ tokens/s per node in large-batch decode (Chapter 13's arithmetic), roughly **4 node-days**; through a commercial API, whatever 3.6B tokens costs on the current price sheet — typically the difference between "a long weekend on idle training hardware" and "a procurement conversation." Two levers dominate the bill: dropping best-of-8 to best-of-4 where a mechanical verifier exists (halves the judge line with no quality loss on verifiable data), and prefix caching on the shared template stem (the persona template above repeats its first ~200 tokens 600K times).

Diversity decays silently within a run. Long generation runs drift toward the generator's favorite phrasings even with good seeds — sample temperature and seed rotation help, but the real defense is measuring: track n-gram novelty and inter-sample similarity *during* the run, and stop or re-seed when the curves flatten. Fifty thousand near-identical "write a function that reverses a string" pairs teach less than five hundred varied ones and cost a hundred times more.

For a bilingual model, generate natively per language. Generating English then translating produces translationese — a collapsed register that RULER-style probes won't catch but native speakers will. Seed personas, topics, and instructions in Spanish and Portuguese directly; use the English teacher for what it is uniquely good at (hard reasoning, code) and verify answers language-independently. Report per-language funnel rates: if the Portuguese survival rate is half the English one, your judge is the problem as often as your generator.

Common pitfalls

Skipping the funnel because "the teacher is strong."

Strong teachers produce confident wrong answers at scale. The funnel is not optional; it is the product.

Judge and generator from the same family.

Self-preference bias silently inflates every score; cross-family judging or answer-verification breaks the mirror.

Benchmark contamination via the teacher.

The eval jump that makes everyone happy is the alarm. Decontaminate before celebrating (§27.4).

Replacing real data instead of augmenting it.

The collapse literature's failure mode, reproduced in production by teams who "cleaned up" the mix (§27.3).

One mega-prompt for everything.

Provenance dies and so does the ability to fix a bad template. One template = one tracked id = one removable slice.

Unbudgeted judges.

Best-of-8 with order-swapped judging costs ~17 inference calls per kept sample. Price it before the run, not after the invoice.

27.7 Decision framework

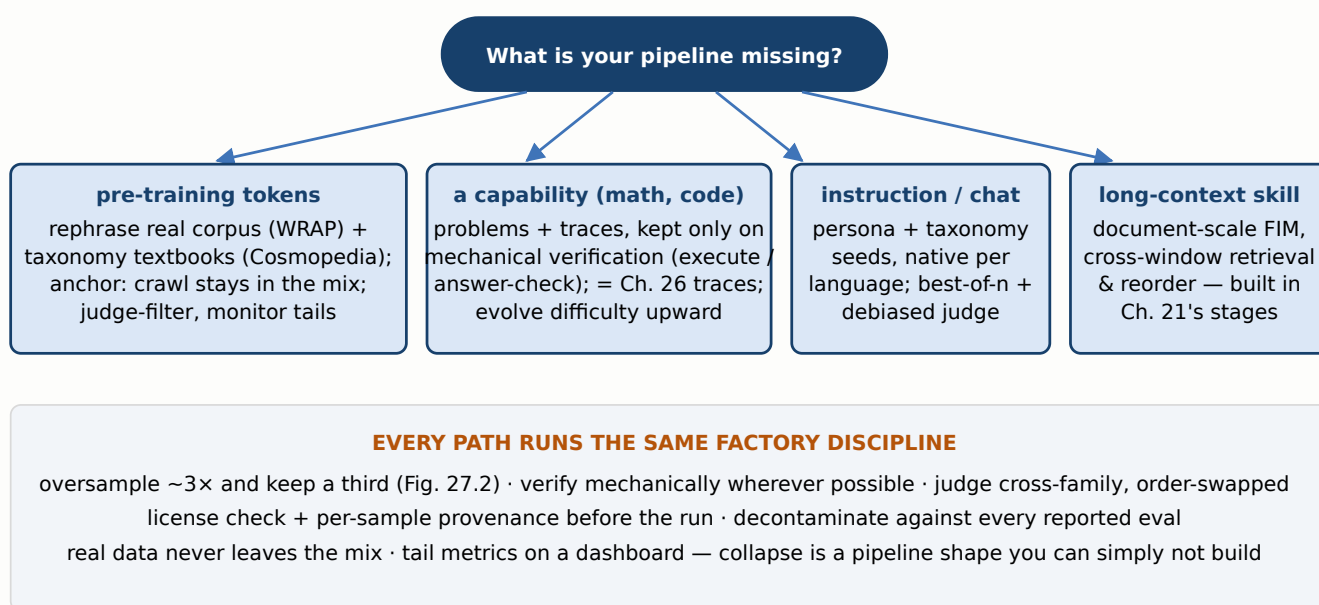


Figure 27.4 — Choosing a synthesis recipe. The gap being filled picks the generation pattern; the discipline underneath is invariant. If a proposed synthetic-data project can't fill in the bottom box, it isn't ready to run.

Cost-benefit, bluntly. Verified capability data (code, math) is the best purchase in post-training: the verifier makes quality compound and Chapter 26 showed the ceiling (R1's 800K traces). Instruction/chat synthesis is nearly as good per dollar but leans on judges — budget the double-scoring and the human audits. Pre-training-scale synthesis is the expensive frontier move: worth it when your real corpus is the binding constraint (small languages, specialized domains — precisely the situation this book's audience is in more often than the frontier is), throwaway when you still have unread crawl. And everywhere, the marginal dollar goes to the funnel before the generator: better filtering on the same candidates beats more candidates through the same filters. **How this scales:** the factory's unit economics improve with everything Chapter 13 and 15 did to inference — and its output is reusable across every model size and run you'll ever train, which makes a curated synthetic set one of the few assets in this book that appreciates.

27.8 Summary

Synthetic data is a factory with three stations, and quality is manufactured at the second one. Generation is a design space, not a method list: seeds buy variety (examples, corpora, personas, taxonomies, or Magpie's nothing-at-all), the generator pattern sets difficulty (imitate, evolve, transform, teach, solve), and the verifier column decides trust — put volume where checking is mechanical. The funnel is the product: oversample $\sim 3\times$, dedup, gate, execute-or-check what can be, judge the rest with a cross-family, order-swapped, length-suspicious judge, and expect to keep about a third. Model collapse is real and avoidable in the same breath: it is what happens when synthetic data *replaces* real data generation after generation; anchor the real corpus, verify what enters, seed for the tails (natively, per language), and monitor diversity, and the photocopy effect never starts. Compliance is a gate — teacher license before the run, provenance on every sample, decontamination against every eval you report — and the labs' shipped pipelines (Phi's synthetic pre-training, Nemotron's 98%-synthetic alignment, Llama's execution-verified loops) all obey it. The strategic fact underneath: for teams whose real-data constraint binds hardest — small languages, specialized domains, modest budgets — the synthetic factory is the great equalizer, and it rewards exactly the discipline this book keeps preaching: measure the funnel, verify what you can, and never let the demo data skip the pipeline.

References

1. Abdin, M., et al. (2024). Phi-4 Technical Report. arXiv:2412.08905. $\sim 400\text{B}$ tokens of curated synthetic pre-training data; synthetic-heavy post-training.
2. NVIDIA (2024). Nemotron-4 340B Technical Report. arXiv:2406.11704. 98% synthetic alignment data; generator/judge/preference kit under a permissive license.
3. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. Rejection sampling for post-training data; execution-feedback loops for code.
4. Wang, Y., et al. (2023). Self-Instruct: Aligning Language Models with Self-Generated Instructions. *ACL*. arXiv:2212.10560. 175 seeds $\rightarrow$ 52K instructions.
5. Xu, C., et al. (2024). WizardLM: Empowering Large Language Models to Follow Complex Instructions (Evol-Instruct). *ICLR*. arXiv:2304.12244.

6. Ge, T., et al. (2024). Scaling Synthetic Data Creation with 1,000,000,000 Personas. arXiv:2406.20094.
7. Xu, Z., et al. (2024). Magpie: Alignment Data Synthesis from Scratch by Prompting Aligned LLMs with Nothing. arXiv:2406.08464.
8. Maini, P., et al. (2024). Rephrasing the Web: A Recipe for Compute and Data-Efficient Language Modeling (WRAP). *ACL*. arXiv:2401.16380.
9. Ben Allal, L., et al. (2024). Cosmopedia. Hugging Face release & report. 25B tokens / 30M documents of open textbook-style synthesis.
10. Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*. arXiv:2306.05685. Position, length, and self-preference biases quantified.
11. Shumailov, I., et al. (2024). AI Models Collapse When Trained on Recursively Generated Data. *Nature* 631. Tail-first degradation under replace-style recursive training.
12. Gerstgrasser, M., et al. (2024). Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. arXiv:2404.01413.
13. Yang, A., et al. (2025). Qwen2.5-1M Technical Report. arXiv:2501.15383. Synthetic long-dependency tasks in the context-extension stages.
14. DeepSeek-AI (2025). DeepSeek-R1. arXiv:2501.12948. 800K verified traces; see Chapter 26.
15. Ding, N., et al. (2023). Enhancing Chat Language Models by Scaling High-Quality Instructional Conversations (UltraChat). *EMNLP*. arXiv:2305.14233.
16. Zhou, C., et al. (2023). LIMA: Less Is More for Alignment. *NeurIPS*. arXiv:2305.11206. 1,000 curated examples align a 65B model; SFT teaches format, pre-training holds the knowledge.

Related: *Chapter 26 — Distillation* (when the generator is a better model, that's black-box distillation) · *Chapter 10 — Data* (dedup and versioning machinery) · *Chapter 9 — evals* (contamination checks) · *Chapter 21 — Extending context* (synthetic long-range dependencies) · *Chapter 29 — the case study*. · [Training Efficient LLMs](#)